Session 4 — LLMs (I): from language models to the Transformer

Artificial Intelligence · 4th year · UIR · Pr. Hakim Hafidi

Study sheet — exam material

1. The thread of this session

This is the first brick of the thread that runs to the end of the course: *predict the next word → an assistant → an assistant that reads your documents → an assistant that acts*.

Everything a large language model does comes from one deceptively small idea: given the text so far, assign a probability to every possible next token. This session takes that idea from counting words in a corpus to the Transformer architecture, and explains why scaling it turned out to be enough to produce systems that write code, translate and pass exams.

Four things to be able to explain by the end: what a language model is, why counting cannot work, how words become numbers that carry meaning, and what attention does.

2. What a language model is

You already use several. Every time your phone finishes your sentence, a search box completes your query, a spell-checker suggests a correction, or a dictation app resolves an ambiguous sound, a language model is answering the same question: *what comes next?*

The definition. A language model assigns a probability to every possible next token given the preceding text. Note what this is *not*: it is not a function returning one word. The output is a distribution over the entire vocabulary — typically 50,000 to 200,000 entries, nearly all of them with probabilities close to zero, summing to one.

An example. Given "I saw a ___", a trained model might produce: *cat* 31%, *dog* 19%, *man* 11%, *movie* 7%, *car* 5%, *bird* 4%, … *banana* 0.06%, *the* 0.01%. Every word in the vocabulary receives a value. Generating text means picking one from this list, appending it, and asking the same question again — a process called **autoregressive generation**, and the reason chat interfaces appear to type their answers.

Why a distribution rather than an answer. Many continuations are plausible. The distribution expresses uncertainty; how we collapse it into one token is a separate decision — the decoding settings you measured in Labs 1 and 2 — and a model that always picks the most probable token produces flat, repetitive text.

The probability of a whole sentence is the product of its next-word probabilities: P("I saw a cat") = P(I) × P(saw | I) × P(a | I saw) × P(cat | I saw a). Every factor is a next-token question. So a language model only ever has to answer one kind of question, again and again.

3. Counting words: n-gram models and why they fail

The Markov assumption. Estimating P(cat | I saw a) from a corpus is easy if you can count. But most long histories never appear twice, so we approximate: assume the next word depends only on the last $n-1$ words. A bigram model looks at one word, a trigram at two, a 5-gram at four.

The model is a table of counts. In a corpus, count how often "saw a cat" appears (say 1,204), how often "saw a" appears in total (say 4,050), and divide: P(cat | saw a) ≈ 0.30. That is the entire model. It was the state of the art in speech recognition and machine translation for decades.

Three fatal limitations.

Sparsity. Most sentences have never been written before. Any sequence absent from the corpus gets a count of zero and therefore probability zero — even when it is perfectly good language. Smoothing techniques patch this, but the underlying problem is that counting cannot generalise to the unseen.

No long memory. "The girl who lived next door to the old baker … was" — was what? The grammatical subject is ten words back, outside any practical window. Enlarging the window makes sparsity exponentially worse.

No notion of meaning. For an n-gram model, "saw a cat" and "saw a dog" share nothing. The words are atomic symbols with no relationship, even though cat and dog are obviously similar. Learning something about one teaches nothing about the other.

What n-gram text looks like. Locally fluent, globally meaningless: every three words make sense, no sentence does. The model has no memory beyond its window and no idea what it is saying.

The conclusion that drives the rest of the session. To generalise, the machine needs to know that *cat* and *dog* are similar. That requires representing words as numbers that carry meaning.

4. Embeddings: turning words into meaning

One-hot vectors, and why they are useless. The obvious encoding gives each word a vector of zeros with a single 1 at its own position. It works as an identifier and fails as a representation: every pair of words is equally distant. Rome and Paris are exactly as different as Rome and banana.

Dense vectors. Instead, represent each word by a few hundred real numbers — a point in a high-dimensional space. Words used in similar contexts end up at nearby points. Rome and Paris share most of their values, because both appear surrounded by the same kinds of words.

An analogy. A personality test scores a person on five traits. Two people with similar scores are similar people. The five scores are a vector: a position in a space. Embeddings do this for words, with a few hundred dimensions instead of five — and nobody decides what the dimensions mean. They emerge from co-occurrence.

Cosine similarity. To compare two vectors we ask whether they point in the same direction: +1 means identical direction, 0 unrelated, −1 opposite. This single operation underlies semantic search, recommendation, and the retrieval you will build in the RAG session.

Meaning becomes geometry. The famous result (Mikolov et al., word2vec, 2013): take the vector for *king*, subtract *man*, add *woman*, and the closest vector in the space is *queen*. Similarly Paris − France + Italy ≈ Rome. The direction from king to queen is the same as the direction from man to woman. Directions in the space correspond to concepts — not because anyone built that in, but because it falls out of predicting which words occur together.

Clusters. Projected down to two dimensions, an embedding space shows vehicles in one region, foods in another, sports elsewhere. The distance an n-gram model could never see is now explicit: cat and dog are neighbours.

Beyond words. The same machinery embeds sentences, paragraphs, documents, images, products, users. Feed in an object, get a vector; similar objects land nearby. This is the engine of semantic search and of retrieval-augmented generation.

5. Tokens: what the model actually reads

Models do not read words. They read **tokens** — sub-word pieces from a fixed vocabulary, produced by an algorithm such as byte-pair encoding that learns the most useful pieces from a corpus.

Examples. "unbelievable" splits into *un* | *believ* | *able*. "ChatGPT" into *Chat* | *G* | *PT*. Common words are a single token; rare words are split into several. A typical vocabulary holds 50,000 to 200,000 pieces.

The rule of thumb. In English, one token is roughly three-quarters of a word, or about four characters. Other languages cost considerably more — French noticeably, Arabic often two to three times English for the same content, because the tokenizer's vocabulary was learned on a corpus dominated by English.

Why this matters practically. Everything is counted in tokens: the price you pay per million, the size of the context window, the speed of generation. A system serving customers in Arabic pays roughly twice as much per conversation as the same system in English, and fills its context window twice as fast. This is not a detail — it is a design constraint for anyone deploying in Morocco, and you measured it yourself in Lab 1.

6. Neural language models

Three steps. *Embed*: each input token becomes a vector. *Process*: a network combines the context into a single representation. *Score*: a final layer plus a softmax turns that into a probability for every token in the vocabulary. The output is the distribution from section 2 — computed by a network instead of counted from a corpus.

Training is self-supervised. Take any text. Hide the next token. Ask the model to predict it. Compare with the truth: for "I saw a ___" the real word was *cat*, and the model gave it 0.31. The loss says: raise *cat*, lower the others. Backpropagation adjusts every weight a little. Repeat on every position of every document. No human labels anything — which is why the training set can be trillions of tokens.

Generation is a loop. Predict, sample a token, append it to the input, predict again. Each output becomes part of the next input. This is what autoregressive means, and it has a consequence worth noting: an error early in a generation conditions everything that follows.

The memory problem of recurrent networks. Before 2017, the processing step was a recurrent network reading one token at a time and carrying a summary forward. Each step overwrites part of that summary, so information from far back fades. And because reading is inherently sequential, training cannot be parallelised across the sequence. Both problems become fatal as sequences and datasets grow.

7. Attention

The idea. Let every token look directly at every other token and decide, for itself, which ones matter.

The standard example. "The animal didn't cross the street because *it* was too tired." What does *it* refer to? The model computes, for *it*, a weight on every other word: high on *animal*, lower on *street*, little elsewhere. Change the last word to *wide* — "because it was too wide" — and the weights flip to *street*. Nobody programmed either behaviour; both emerge from training.

Query, key, value. The mechanism is a soft lookup. Each token produces a *query* (what am I looking for?), and every token offers a *key* (what do I have?) and a *value* (what I hand over). The query is compared with every key — a dot product, closely related to the cosine similarity of section 4 — the scores are turned into weights that sum to one, and the output is the weighted average of the values. Every token does this with respect to every other token.

Two consequences. Distance stops mattering: a token can attend directly to one a thousand positions earlier, at the same cost as its neighbour. And nothing is sequential, so the whole sequence is processed in parallel — which is what makes training at scale on GPUs possible.

Multi-head attention. Several attention mechanisms run side by side, each learning a different kind of relation: one tracks grammatical subject, another local context, another topic. GPT-3 uses 96 heads per block. Their outputs are combined. Again, nobody specifies the relations; they emerge from next-token prediction.

8. The Transformer

One block. A Transformer block has two parts. *Self-attention*: each token gathers information from the others. *Feed-forward*: each token independently processes what it gathered. Stack the block N times — 12 in GPT-2, 96 in GPT-3 — and deeper blocks capture more abstract patterns: grammar first, then facts and relationships.

Positional encoding. Attention treats its input as a set, so order is invisible to it. "The dog bit the man" and "The man bit the dog" would be identical. The fix is to add to each token a vector that depends on its position, so that order is part of the representation.

Encoders and decoders. An *encoder* lets every token see every other token, left and right: best for understanding — classification, search, extraction. BERT (2018) is the archetype, and it cannot generate text. A *decoder* lets each token see only what came before: best for generating, one token at a time. Every chat model is a decoder. The name GPT spells it out: **G**enerative **P**re-trained **T**ransformer.

The paper. *Attention Is All You Need*, Vaswani et al., Google, 2017. The title is a claim: recurrence is unnecessary.

9. Pretraining at scale

What pretraining is. Next-token prediction over an enormous corpus — a large slice of the public internet, books, code. Self-supervised, so no labelling. The result is a **base model**.

GPT-3 in numbers (2020). 175 billion parameters. About 300 billion training tokens. A context window of 2,048 tokens. 96 Transformer blocks, 96 attention heads per block. Estimated compute cost around 4.6 million dollars. Every one of those numbers has grown since; the principle has not.

Scaling laws (Kaplan et al., 2020). Across seven orders of magnitude of compute, the prediction error falls smoothly and predictably as model size, dataset size and compute increase — with no plateau in sight at the scales tested. This is why laboratories kept building larger models rather than waiting for better ideas. The Chinchilla result (DeepMind, 2022) refined it: for a fixed budget, data matters as much as parameter count, and earlier models were badly under-trained on data.

What a base model actually does. It completes text. Prompted with "Write a poem about the sea", a base model may well continue "Write a poem about the mountains. Question 3: describe your summer holidays in 200 words." It has seen millions of exam papers, so it produces the next exam question. It has read the internet and learned its patterns — facts, styles, code, dozens of languages — but it has no notion that it is being *asked* something.

Turning a completer into an assistant is Session 5.

The cost. Training GPT-3 cost roughly 5 million dollars in 2020; GPT-4 reportedly around 100 million in 2023; frontier runs in 2025–26 are estimated above a billion. Tens of thousands of GPUs for months, drawing power comparable to a small town. A handful of organisations can afford it — which is why open-weight models matter, and why most engineering work consists of using these models rather than building them.

10. Key terms

Term	Meaning
Language model	Assigns a probability to every possible next token given the preceding text.
Autoregressive generation	Predict, sample, append, repeat — one token at a time.
N-gram model	Estimates the next word from counts of the last n−1 words (Markov assumption).
Markov assumption	The next word depends only on a fixed number of preceding words.
Sparsity	Most sequences never appear in the corpus, so counting assigns them probability zero.
One-hot vector	A vector with a single 1; an identifier carrying no information about meaning.
Embedding	A dense vector of real numbers representing a token, sentence or document; distance encodes similarity.
Cosine similarity	+1 same direction, 0 unrelated, −1 opposite; the standard way to compare embeddings.
word2vec	The 2013 method that popularised embeddings and the king − man + woman ≈ queen result.
Token / tokenization / BPE	Sub-word pieces the model actually reads; byte-pair encoding learns the vocabulary.
Softmax	Converts raw scores into probabilities summing to one.
Recurrent network (RNN)	Reads a sequence step by step carrying a memory; forgets long contexts, trains sequentially.
Attention	Each token weighs every other token when computing its representation.
Query / key / value	What I am looking for / what each token offers / what it hands over.
Multi-head attention	Several attention patterns in parallel, each capturing a different relation.
Positional encoding	A position-dependent vector added to each token so order is not lost.
Transformer block	Self-attention plus feed-forward, repeated N times.
Encoder vs decoder	Sees both directions (BERT, understanding) vs only the past (GPT, generation).
Pretraining	Self-supervised next-token training at scale.
Base model	The result of pretraining: completes text, does not follow instructions.
Scaling laws	Loss falls predictably as compute, data and parameters grow (Kaplan 2020; Chinchilla 2022).

11. Numbers to know

GPT-3 (2020): 175 billion parameters · 300 billion training tokens · 2,048-token context · 96 blocks · 96 attention heads per block · ~$4.6 M compute. Vocabulary: 50,000–200,000 tokens · 1 token ≈ ¾ of an English word ≈ 4 characters · Arabic typically 2–3× the tokens of English for the same content. Transformer paper: 2017. word2vec: 2013. BERT: 2018. Chinchilla: 2022. Frontier training runs 2025–26: estimated above $1 billion.

12. Common misconceptions

"The model looks up answers in a database." There is no database. There are weights, and a forward pass that computes a distribution. This is why a model can produce a perfectly formatted citation that does not exist.

"Bigger context window means the model uses all of it well." Fitting in the window and being used well are different things. Models reliably overlook facts buried in the middle of long inputs.

"Embeddings understand meaning." They encode statistical co-occurrence. That turns out to behave remarkably like meaning for many tasks, and to diverge from it in ways that matter — including inheriting the biases of the corpus.

"Tokens are words." They are sub-word pieces. This is why a model can struggle to count the letters in a word: it never sees the letters.

13. Pause & recall — answers

I. (a) A function assigning a probability to every possible next token given the previous ones. (b) Because many continuations are plausible; the distribution expresses uncertainty and sampling picks one. (c) Search autocomplete, keyboard suggestions, translation, speech recognition, chatbots.

II. (a) That the next word depends only on the last few words. (b) Because sequences never seen in the corpus have a count of zero. (c) Locally fluent means any three consecutive words make sense; globally meaningless means the whole text has no coherent content.

III. (a) Because all one-hot vectors are equally distant: Rome and Paris are as different as Rome and banana. (b) That directions in the space encode relations, not just proximity. (c) A sub-word piece; prices, context limits and speed are all counted in tokens, and non-English text uses more of them.

IV. (a) Embed the tokens; process the context; score every vocabulary token with a softmax. (b) Because each sampled token is fed back as input before the next prediction. (c) Long-range dependencies fade in recurrent networks; attention lets any token look directly at any other, and trains in parallel.

V. (a) Self-attention and a feed-forward network. (b) Because attention treats its input as a set, so without positional information "the dog bit the man" equals "the man bit the dog". (c) A pretrained text completer; it does not follow instructions or answer questions — that requires the fine-tuning of Session 5.

14. Sample exam questions

1. Explain why an n-gram model cannot generalise, and how embeddings address this.
2. Describe attention using the "it was too tired" example, then state what a Transformer block contains.
3. What is a base model, and why is pretraining called self-supervised?
4. A Moroccan company plans an Arabic-language assistant. Explain, in terms of tokenization, what this costs compared with English and what design consequences follow.
5. State the scaling laws and explain what the Chinchilla result changed.